Winter Quarter NDSC Full Project Timeline

Winter Quarter NDSC Full Project Timeline

Week 3: First Meeting

Week 4: Initial EDA (Exploratory Data Analysis)

Week 5: Data Cleaning and Feature Engineering

Week 6: Final EDA and Modeling

Week 7: Model Interpretation & Insights

Week 8: Presentation and Dashboard Building

Week 9: Project Showcase

Week 3 Meeting

Week 3: First Meeting

Introductions and Introducing Project Idea

Group Introductions:

- Name
- Major
- Year
- What is a hobby you like to do?
- What do you know about the topic of our project? (Machine Learning)
- What do you want to learn or are most excited about for this project?

Meeting Time

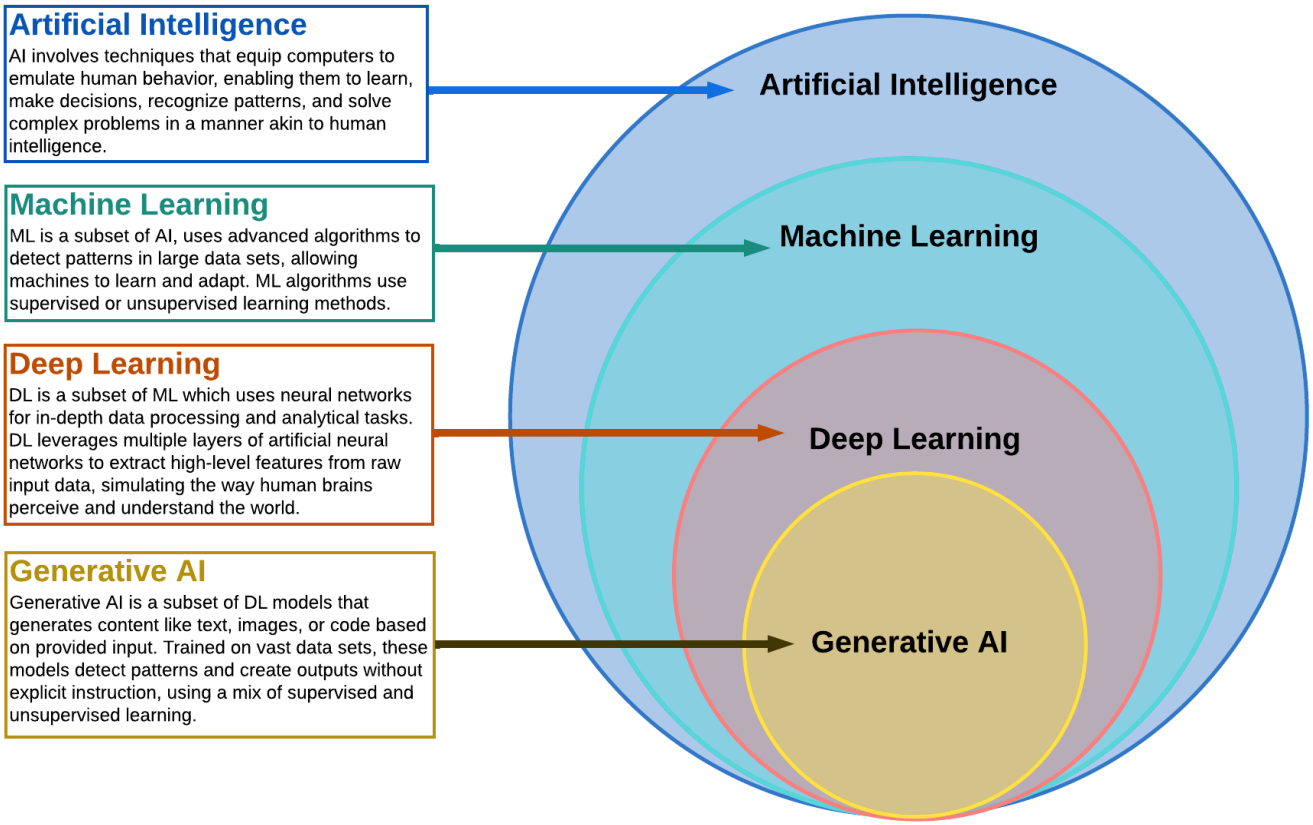
Wednesday evening 8:30pm

iMessage for communication

Introduction to Machine Learning

Machine Learning (ML):

- A subset of Artificial Intelligence (AI) where machines learn patterns from data to make predictions or decisions without being explicitly programmed for every rule.
- Key idea: Instead of writing rules ("If income < 50k, flag as high risk"), the machine learns from examples.



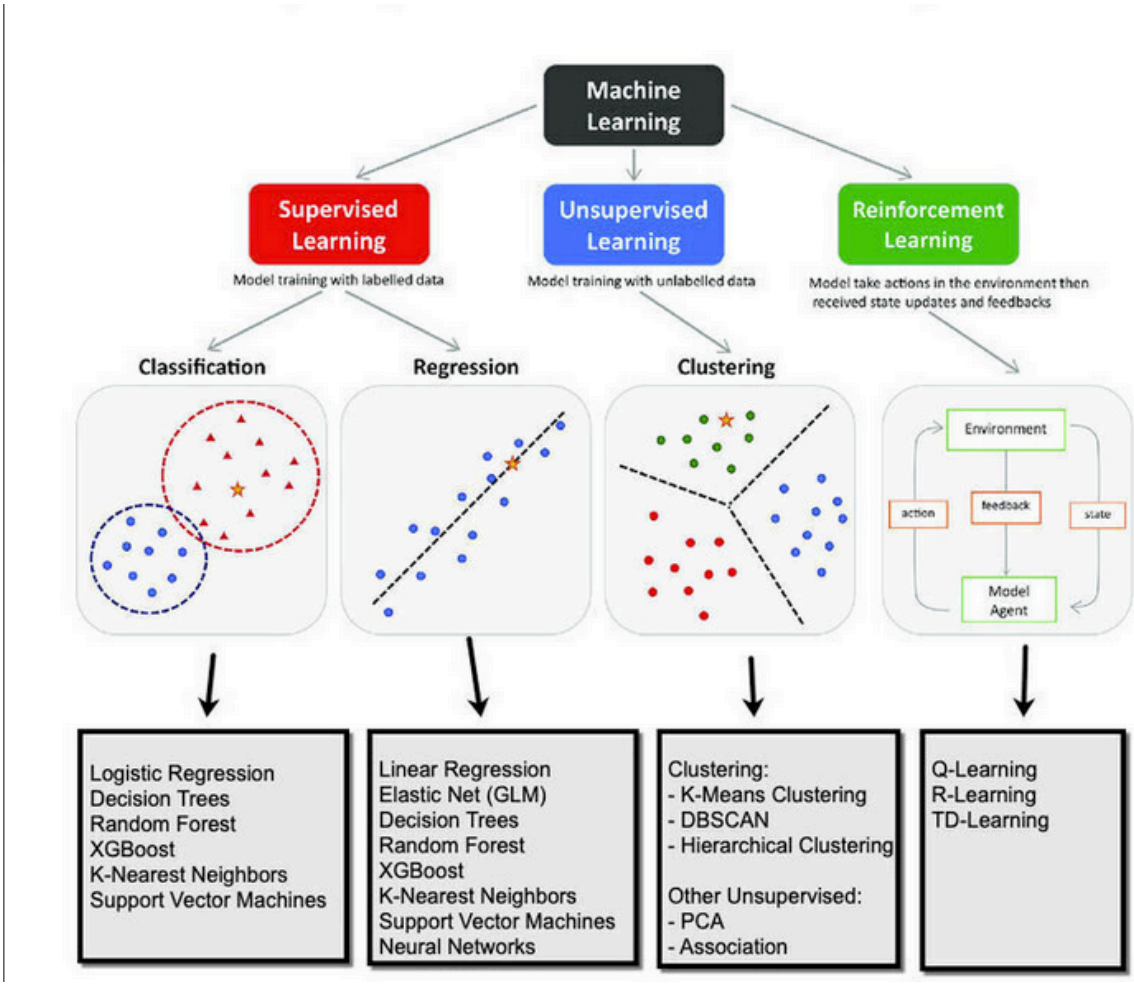
Unraveling AI Complexity - A Comparative View of AI, Machine Learning, Deep Learning, and Generative AI.

(Created by Dr. Lily Popova Zhuhadar, 07, 29, 2023)

AI vs. ML

Term	Definition	Example
AI (Artificial Intelligence)	Broad field where machines perform tasks requiring "intelligence"	ChatGPT answering questions, self-driving cars
ML (Machine Learning)	Subset of AI that **learns from data**	Predicting song yearly streams, classifying emails as spam/not spam
Deep Learning	Subset of ML using neural networks	Image recognition, speech-to-text

ML is a **tool in AI**, not all AI is ML.



Types of Machine Learning

Supervised Learning

- Definition: Learn a mapping from input features to known outputs.
- Input: Features (X)
- Output: Labeled data (Y)
- Goal: Predict outcomes for new/unseen data.
- Examples:
    - Identifying if a picture contains a cat or not (Yes/No)
    - Predicting house prices
- Algorithms: Linear Regression, Logistic Regression, Decision Trees, K-Nearest Neighbors, Random Forest, Gradient Boosting, etc.

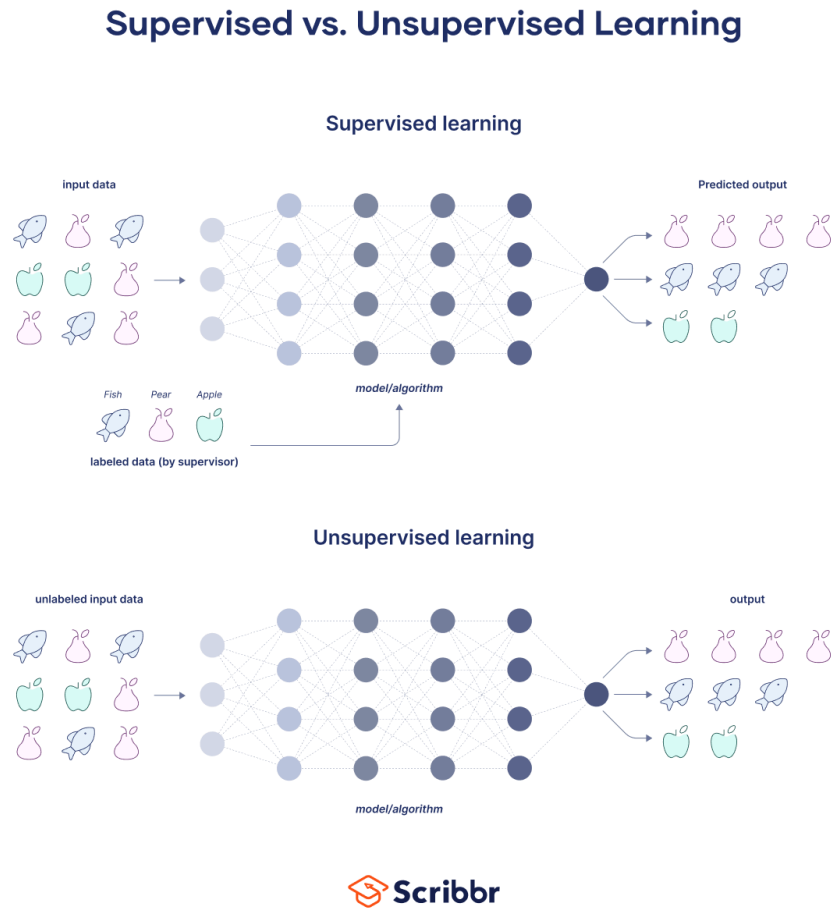
Unsupervised Learning

- Definition: Find patterns or structure in data without labels.
- Input: Features (X)
- Output: Clusters, patterns, or dimensions

- Goal: Explore data, detect anomalies, or reduce dimensions.
- Examples:
    - Grouping customers by spending habits
    - Detecting unusual transactions (fraud)
    - Market basket analysis (which products are frequently bought together)
- Algorithms: KMeans Clustering, DBSCAN, PCA

Reinforcement Learning

- Definition: Learn by trial and error, receiving rewards or penalties.
- Goal: Maximize cumulative reward over time.
- Example:
    - Teaching a robot to walk
    - AI playing video games (chess, etc.)
    - Trading bots that learn when to buy/sell

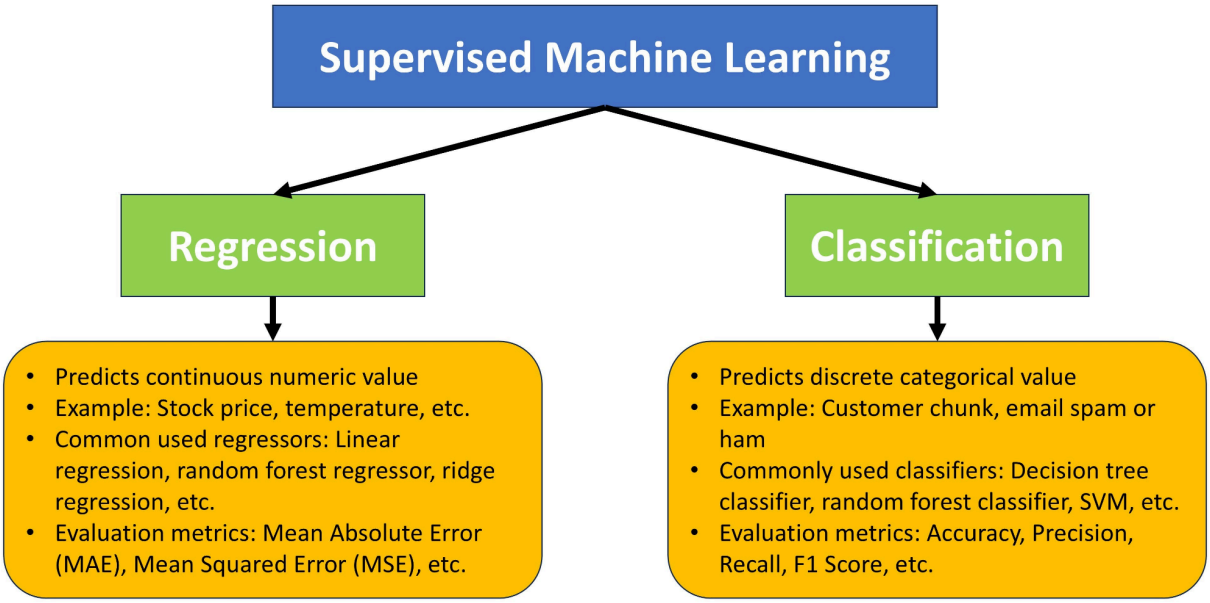


Supervised Learning vs. Unsupervised Learning

Aspect	Supervised	Unsupervised
Data	Labeled (features + target)	Unlabeled (features only)
Goal	Predict outcomes / labels	Discover patterns / structure
Example	Predict movie audience count	Segment customers for targeted marketing
Algorithms	Linear Regression, Logistic Regression, Random Forest, etc.	KMeans Clustering, PCA, DBSCAN, etc.
Metrics	R^2, RMSE, MAE, Accuracy, ROC-AUC, Precision/Recall	Silhouette score, Davies-Bouldin index

Analogy:

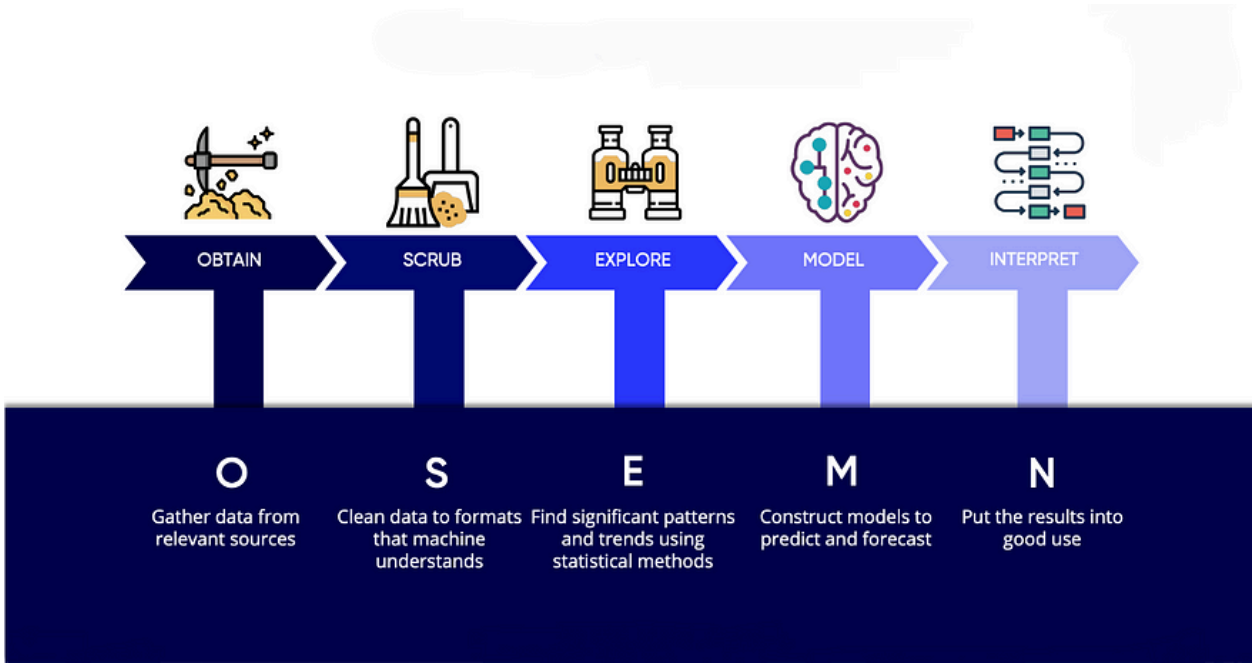
- Supervised = professor gives you the answers to learn from
- Unsupervised = professor just gives the homework without answers; you look for patterns



Classification vs. Regression

Aspect	Classification	Regression
Output	Categories / Classes	Continuous numbers
Example	Predict if email is spam or not spam (Yes/No)	Predict annual income, house price, stock price
Metrics	Accuracy, Precision, Recall, ROC-AUC	R^2, RMSE, MAE
Algorithms	Logistic Regression, Decision Tree Classifier, Gradient Boosting, etc.	Linear Regression, Decision Tree Regressor, Gradient Boosting Regressor, etc.

Machine Learning Project Pipeline



Machine Learning Project Pipeline
This pipeline is the roadmap for any ML project, from raw data to actionable insights.

Obtain Data

- Goal: Get the data you'll use to train and evaluate your models.
- Tasks:
    - Download from public sources (Kaggle, UCI, company database)
    - Understand the columns/features and target variable
    - Check the size of the dataset and if it's suitable for your models

Tip: Use .head() and .info() in Python to better understand your data.

Clean and Preprocess Data

- Goal: Make the data ready for modeling. Raw data often has errors, missing values, or inconsistent formatting.
- Tasks:
    - Handle missing values (drop, fill with mean/median/mode, or use algorithms that handle missing data)
    - Encode categorical variables (e.g., home_ownership → one-hot encoding)
    - Convert text / strings to numeric values if needed
    - Scale or normalize numeric features if required (especially for distance-based models)

    - Remove irrelevant features that won't help the model

Tip: Garbage in → garbage out: a model can only be as good as the data it's trained on.

Explore Data

- Goal: Understand your data patterns, relationships, and distributions before modeling.
- Tasks:
    - Create data visualizations of your data to try and find patterns
    - Analyze correlations between features and the target variable
    - Look for outliers or unusual patterns

Tip: This is where you can start making observations and hypotheses: "Users with higher activity levels appear to have better outcomes than less active users."

Model Generation

- Goal: Train ML models to predict outcomes.
- Steps:
    1. Split data into training and testing sets (e.g., 80/20)
    2. Choose model(s): logistic regression, decision tree, random forest, gradient boosting
    3. Train models on training data
    4. Evaluate on testing data using appropriate metrics

Tip: Baseline models help us understand the simplest prediction, then we can try more complex models.

Interpret the Model

- Goal: Understand what the model learned and how features affect predictions.
- Tasks:
    - Feature importance → which features drive predictions most
    - Coefficients for linear models → direction and strength of influence
    - Analyze errors → where does the model fail (false positives / false negatives)
    - Tie results back to business insights

Tip: Models are tools to support decisions, not just numbers.

Pipeline Summary

1. Obtain Data: Download & understand dataset
2. Clean & Preprocess: Fix missing values, encode, scale
3. Explore Data: Plot distributions, analyze correlations

4. Model Generation: Train & evaluate supervised models
5. Interpret Model: Feature importance, business insights, actionable recommendations

Big Picture: The steps don't always go in order, exploring the data can reveal issues that require earlier fixes.

Project Idea

Project Idea: Predicting Credit Risk & Loan Defaults

Domain Knowledge Overview

What Is Credit Risk?

Credit risk means: The chance that someone will not pay back the money they borrowed.
Banks and lenders care about credit risk because they lend money to make money.
If borrowers do not repay their loans, the bank loses money.

Why Credit Risk Matters

When borrowers default on loans:

- Banks lose money
- Lending becomes riskier

Credit risk affects:

- Interest rates: Riskier borrowers are often charged higher interest
- Loan approval decisions: Some applicants may be denied loans
- Capital allocation: Banks must keep extra reserves for risky loans

Simple Example

Imagine lending money to a friend:

- You lend your friend $10,000
- If they repay → the risk was low, and you earn interest
- If they do not repay → you lose the $10,000 and the chance to use it elsewhere

Banks make this decision thousands of times, so they use data to help decide.

What Is a Loan Default?

A loan default happens when a borrower stops paying their loan.
This can include:

- Missing multiple monthly payments
- Only paying part of what they owe (charged-off loans)
- Bankruptcy that affects repayment

For our machine learning project, we simplify this into two outcomes:

- 0 = Fully Paid
- 1 = Default / Charged Off

Why This Is a Machine Learning Problem
We want to answer one simple question:
Based on past data, will this borrower probably pay back the loan?
Because we know the outcome for past borrowers, this becomes a supervised learning classification problem where the model predicts:

- Yes (will repay)
- No (will default)

What Information Does the Model Use?
The model looks at information that helps describe a borrower's ability to repay.
Examples include:

- How much money they are borrowing
- How long they have to repay the loan
- How much money they earn
- How much debt they already have
- How stable their job is
- How they have handled money in the past

These factors help the model learn patterns linked to repayment behavior.

Understanding Ratios
Instead of focusing on formulas, think about affordability:
If someone earns $4,000 per month but must pay $2,500 every month toward loans, they have very little money left for emergencies.
This situation is considered higher risk.

How Banks Can Use Credit Risk Predictions
Banks use model predictions to:

- Approve or deny loans
- Decide how much interest to charge
- Identify loans that need extra review

Important note:
Machine learning models support human decisions, they do not replace humans.

Why Machine Learning Is Useful Here
Machine learning is helpful because it:

- Can analyze many factors at once
- Can detect patterns humans might miss
- Produces probabilities, not just yes/no answers
- Supports data-driven decision-making

Credit Risk Machine Learning Project Workflow

1. Obtain Data
   Download a real dataset
2. Clean and Preprocess Data
   Handle missing values and prepare features for modeling
3. Explore Data
   Visualize patterns and relationships in the data
4. Model Generation
   Train classification models such as Logistic Regression, Random Forest, or Gradient Boosting, etc.
5. Interpret the Model
   Understand which features matter most and what the results mean

Example Insight
After training a model, we might find:

- Certain features consistently appear in high-risk predictions

Action:

- Flag these cases for closer human review rather than automatic decisions

Key Takeaways

- Credit risk is the chance someone will not repay borrowed money
- Predicting loan default is a classification problem
- Good features should make logical sense
- Machine learning helps guide decisions, not replace them

Summary
We use past data about borrowers to train a model that estimates how likely someone is to repay a loan.

Find the Dataset
https://www.kaggle.com/datasets/wordsforthewise/lending-club
https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data

Action Items for Next Week:

- Find some more datasets
- Each person explores one dataset and finds **two to three** observations or questions about the data. This is to get more familiar with the dataset.

Week 4 Meeting

Week 4: Initial EDA (Exploratory Data Analysis)

Dataset Check-In:

- Did you find any datasets that could help our project?
- Which dataset did you look at?
- What were 2-3 interesting things you found about the dataset you looked at?

New Datasets:

- SBA Loan Dataset
  - **Dataset Overview**
    - This dataset is a comprehensive record of **U.S. Small Business Administration (SBA) loan data** spanning from 1987 to 2014. It tracks the lifecycle of loans guaranteed by the federal government to support small business growth.
  - **Content and Variables**
    - The dataset consists of approximately 899,164 observations and 27 variables, which can be grouped into the following categories:
    - **Business Identifiers:** Includes the business name, city, state, and NAICS codes (North American Industry Classification System) to identify the specific economic sector.
    - **Operational Details:** Indicates whether the borrower is a New Business or an Existing Business, the number of employees, and whether the business is female- or minority-owned.
    - **Loan Specifications:** Details the Disbursement Date, the total Loan Amount, the specific bank (Lender) that issued the loan, and the Term (length of the loan in months).
    - **Government Guarantee:** Tracks the SBA's Guaranteed Amount, which is the portion of the loan the government agrees to repay the lender if the business defaults.
    - **Outcome Labels (Target):** The primary indicator of success or failure is the MIS_Status, which categorizes loans as either "P I F" (Paid in Full) or "CHGOFF" (Charged Off/Defaulted).
- Lending Club Company Credit Risk & Population Stability
  - **Dataset Overview**
    - This dataset is specifically designed for Model Monitoring and the study of Population Stability. This source tracks individual consumer loans and is structured to show how credit risk models perform as time passes and economic conditions shift.
  - **Content and Variables**

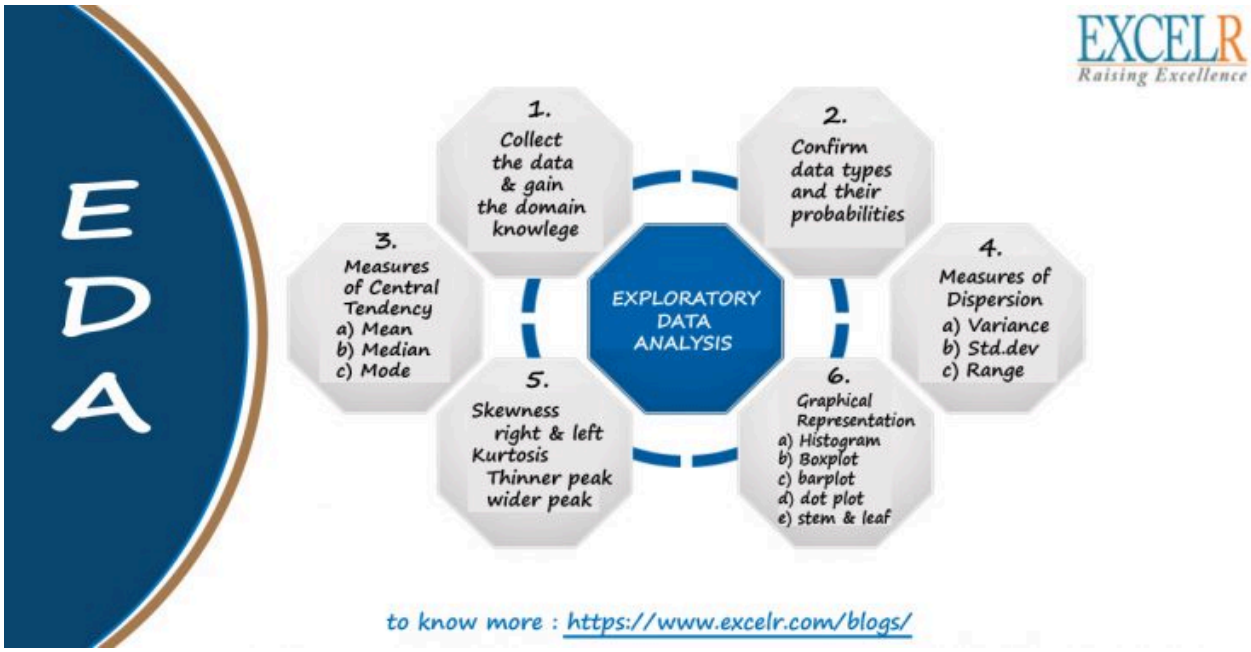
- The dataset is provided in multiple CSV files, most notably split into a large training set (2014–2018) and a more recent testing set (2019–2020). It contains the following types of information:
- **Borrower Financial Profile**: Detailed personal financial markers including annual income, employment length, and home ownership status (Rent, Own, Mortgage).
- **Loan Characteristics**: Includes the loan amount, the interest rate assigned, the loan grade (a risk rating from A to G), and the purpose of the loan (e.g., debt consolidation, home improvement).
- **Credit History**: Variables that track the borrower's past behavior, such as the number of delinquencies in the past two years and the number of inquiries on their credit report.
- **Risk Metrics**: Provides pre-calculated fields used in professional banking, such as Debt-to-Income (DTI) ratios.
- **Outcome Labels (Target)**: The primary target variable indicates the loan status, distinguishing between "Fully Paid" and "Charged Off" (default).

- https://www.kaggle.com/datasets/laotse/credit-risk-dataset/data
- https://www.kaggle.com/datasets/programmer3/credit-risk-dataset

SBA Loan data for our dataset for our project

What is EDA (Exploratory Data Analysis)?

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset before building any machine learning models.



Instead of immediately asking the data to make predictions, EDA focuses on:

- Understanding what the data contains
- Identifying patterns and relationships
- Detecting errors or inconsistencies
- Deciding how the data should be prepared for modeling

EDA is the step where we explore and understand our data so that later modeling decisions are informed, not random.
EDA is not about finding final answers, it is about asking better questions.

Why Is EDA Important?
EDA is one of the most important steps in any data science or machine learning project because models learn directly from the data they are given.

If the data is:

- Messy
- Incomplete
- Misleading
- Poorly understood

Then even the best model will perform poorly.

EDA helps us:

- Catch data problems early
- Understand what features may be useful
- Choose appropriate models and metrics
- Avoid incorrect conclusions
- Explain our results clearly to others

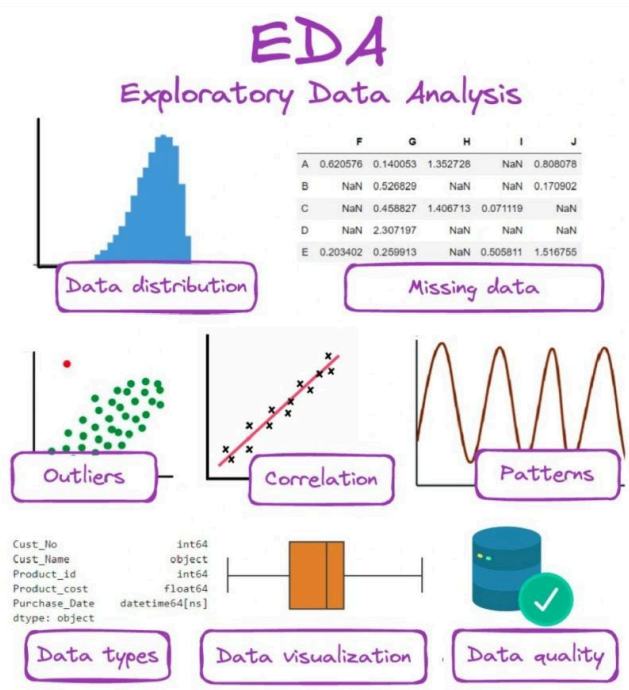
Key idea: If you don't understand your data, your model won't either.

Skipping EDA often leads to:

- Models that seem "accurate" but fail in real life
- Confusing results that are hard to explain
- Wasted time fixing problems later

Core Components of EDA

EDA is made up of several methods that help us understand different aspects of the data. These methods work together and are often repeated as we learn more.



1. Understanding the Dataset Structure

What This Involves

- Number of rows (observations)
- Number of columns (features)
- Data types (numeric, categorical, text)
- Identifying the target variable

Why This Matters

- Some models require numeric inputs
- The number of observations affects model reliability
- Knowing the target tells us whether this is a classification or regression problem

Questions to Ask

- What is each row representing?
- What does each column mean?
- Which variable are we trying to predict?

2. Summary Statistics

What This Involves

- Mean, median, minimum, maximum
- Standard deviation
- Quartiles (25%, 50%, 75%)

Why This Matters

- Provides a quick overview of typical values
- Helps identify extreme or impossible values
- Reveals skewed distributions

Example Insight

If the average value is much higher than the median, it may indicate that a few extreme values are pulling the mean upward.

3. Univariate Analysis (One Variable at a Time)

What This Involves

Analyzing individual features independently to understand their distributions.

Common Methods

- Histograms (numeric features)
- Boxplots (numeric features)
- Bar charts (categorical features)

Why This Matters

- Shows how values are distributed
- Identifies outliers
- Reveals whether values are evenly spread or concentrated

Questions to Ask

- Are most values clustered together?
- Are there extreme values?
- Are some categories very rare?

4. Target Variable Analysis

What This Involves

Understanding how the target variable is distributed.

Why This Matters

- Class imbalance affects model evaluation
- Baseline performance depends on target distribution
- Helps choose appropriate metrics

Example

If 90% of outcomes belong to one class, accuracy alone may be misleading.

5. Bivariate Analysis (Relationships Between Variables)

What This Involves

Examining how two variables relate to each other, especially:

- Feature vs target
- Feature vs feature

Common Methods

- Scatter plots
- Boxplots grouped by target
- Bar charts split by category
- Correlation matrices

Why This Matters

- Helps identify predictive features
- Reveals relationships not visible from single-variable analysis
- Detects redundancy between features

Important Note

Correlation shows association, not causation.

6. Missing Data Analysis

What This Involves

- Identifying which features have missing values
- Measuring how much data is missing
- Understanding patterns of missingness

Why This Matters

- Missing data can bias results
- Some models cannot handle missing values
- The way missing data is handled affects model performance

Questions to Ask

- How much data is missing?
- Is missingness random or systematic?
- Does missing data correlate with the target?

7. Outlier Detection

What This Involves

Identifying values that are unusually large or small compared to the rest of the data.

Why This Matters

- Outliers can distort models
- Some outliers represent real but rare cases
- Decisions must be made whether to keep, cap, or remove them

Common Methods

- Boxplots
- Visual inspection
- Statistical thresholds

Key Idea

Not all outliers are mistakes — some contain important information.

8. Correlation and Feature Relationships

What This Involves
Analyzing how numeric features relate to each other.
Why This Matters

- Highly correlated features may be redundant
- Some models are sensitive to multicollinearity
- Helps simplify models

Caution
Strong correlation does not mean one variable causes another.

9. Hypothesis Formation
What This Involves
Using observations from EDA to form testable ideas.
Example Hypotheses

- Certain groups behave differently than others
- Higher values of a feature may be associated with worse outcomes
- Some features may not add much predictive value

EDA produces questions, not final conclusions.

EDA is iterative, it is not a one-time step but a cycle.

As you explore:

- You may uncover data quality issues
- You may need to return to cleaning
- New questions may arise

Key Takeaway: EDA is the foundation of every successful machine learning project.
It ensures that models are built on understood, trustworthy data, not assumptions.

Data Visualizations
Deep Dive: How to Create Effective Data Visualizations

1. Start with a Question
Definition: Every visualization should have a clear purpose: it answers a question.
Why it matters: Plots without questions are confusing, misleading, or cluttered.
Do:

- Write down the question first: “How does default rate change with loan amount?”
- Make the plot only to answer that question.
- Think about what the audience should take away.

Don’t:

- Plot everything just because it exists.

- Let curiosity alone guide your visualizations — it can lead to irrelevant charts.

Example:

- Good: “What is the distribution of loan amounts for borrowers who defaulted?”
- Bad: “Plot loan amounts for all loans, defaulted or not, without context.”

2. Choose the Right Chart Type

Definition: Matching the chart type to your data type and question is critical.

Data Goal	Chart Type	Dos	Don’ts
Compare categories	Bar chart	Sort bars by value; label axes	Use pie charts with many categories; don’t rotate bars unnecessarily
Show distribution	Histogram/ Boxplot	Choose reasonable bin size; use boxplots to see outliers	Make bins too small or large; use 3D histograms
Relationship between variables	Scatter plot	Use color/size to add dimension; annotate interesting points	Overplotting with thousands of points without alpha; connecting points unless ordered
Trends over time	Line chart	Smooth curves for noisy data if needed; label time axis clearly	Use disconnected lines; clutter multiple lines without legend
Correlation / patterns	Heatmap / Pair plot	Use color gradient to show magnitude; annotate significant values	Use rainbow color map; overcrowding with too many variables

3. Highlight the Important Message

Definition: The key insight should be immediately visible.

Do:

- Use contrasting colors for key data points (e.g., red for high risk)
- Annotate points of interest (“Top 5 default loans”)
- Sort bars/plots to guide the eye

Don’t:

- Make everything the same color
- Rely on the audience to figure out the story themselves

Example:

Highlighting high debt-to-income ratio borrowers in red draws attention to risk patterns.

4. Simplify for Clarity

Definition: Keep plots readable and avoid clutter.
Do:

- Remove unnecessary gridlines or borders
- Limit categories to ≤ 5–7 when possible
- Keep titles concise, descriptive, and meaningful

Don't:

- Include too many colors, lines, or variables
- Add 3D effects unless needed for dimensionality

Example:
Instead of a 10-category pie chart, use a sorted bar chart for clarity.

5. Axes and Scale
Definition: Correct scales prevent misinterpretation.
Do:

- Use consistent scales across similar plots
- Start Y-axis at 0 for meaningful comparisons
- Use log scales for highly skewed data

Don't:

- Truncate axes to exaggerate differences
- Use different scales for similar charts without explanation

Example:
Comparing loan amounts with a Y-axis starting at $10,000 would exaggerate small differences.

6. Color and Style
Definition: Color communicates meaning but can confuse if misused.
Do:

- Use color to encode meaning (default vs non-default)
- Use accessible palettes for colorblind viewers
- Limit color variety (<5 colors per plot)

Don't:

- Use rainbow color maps for continuous variables
- Use too many colors or random colors

Example:
Blue for safe loans, red for risky loans. Avoid green, yellow, pink together.

7. Labeling
Definition: Labels clarify and prevent misinterpretation.
Do:

- Label axes with units: "Monthly Income ($)"
- Provide legends and titles that are self-explanatory

- Use captions to explain the insight

Don't:

- Assume the audience knows abbreviations
- Forget to label axes or categories

Example:
Bad: Title = "Loans"
Good: Title = "Default Rate by Loan Amount Category"

8. Interaction (Optional but Powerful)
Definition: Interactive plots let users explore data themselves.
Do:

- Add filters, dropdowns, sliders
- Use hover info to show extra details
- Update plots dynamically based on user input

Don't:

- Make interactions overly complex for beginners
- Rely solely on interaction to explain the insight

Example:
A slider for loan term (36 vs 60 months) that updates default rate automatically.

9. Iterative Process
Definition: Good visualizations require multiple drafts.
Do:

- Make a first plot
- Ask: Does it answer the question clearly?
- Refine labels, colors, annotations, and scale
- Share with others for feedback

Don't:

- Make a plot once and assume it's perfect

Example:
Try three versions of a boxplot with different colors and annotations; pick the one easiest to read.

10. Storytelling
Definition: Visualization should tell a logical, simple story.
Do:

- Sequence plots in logical order
- Add captions with insights ("Higher DTI → higher default rate")
- Combine multiple visualizations for context

Don't:

- Dump all plots with no explanation

- Assume the audience will interpret patterns themselves

Example:

- Histogram of loan amounts → Boxplot of default vs DTI → Feature importance chart → Dashboard panel

11. Common Pitfalls

Pitfall	How to Avoid
Too much data	Aggregate or filter, focus on what matters
Misleading scales	Always check axes and units
Overplotting	Use transparency (alpha) or sample points
Ignoring outliers	Identify, investigate, and decide to remove or transform
Poor color choice	Use colorblind-friendly palettes; avoid rainbow gradients

12. Visualization Workflow

1. Pick one question
2. Decide chart type
3. Make first plot (Python: matplotlib, seaborn, pandas, plotly)
4. Refine colors, labels, axes
5. Annotate key insights
6. Repeat for each important question
7. Combine into a story (slides or dashboard)

13. Checklist for a Good Visualization

- ☐ One clear question
- ☐ Appropriate chart type
- ☐ Highlight key insight
- ☐ Minimal clutter
- ☐ Correct axes and scale
- ☐ Clear labels and title
- ☐ Meaningful color use
- ☐ Accessible and readable
- ☐ Supports storytelling

Action Items for Next Week:

- Data Cleaning Check:
    - Identify missing values and note patterns
    - Flag extreme or unusual values
    - Feature Summaries:
        - Calculate basic statistics (mean, median, min, max, quartiles)
        - Note skewed distributions or unusual patterns
- Target Analysis:
    - Count defaults vs fully paid
    - Check for class imbalance
- Relationships Between Features:
    - Check correlations between numeric features
    - Compare feature distributions by target variable
- Hypotheses / Observations:
    - Write 1–2 sentences about patterns or insights
    - Example: “Borrowers with high DTI are more likely to default”
- Visualizations:
    - Have at least one data visualization from your exploratory data analysis and a Include a short interpretation in 2-3 sentences of what the plot is and explaining what it shows
- Document Everything:
    - Notes, plots, and observations go into your project folder/report

Week 5 Meeting

Week 5: Data Cleaning and Feature Engineering

Initial EDA Check-In:

- Dataset Overview
  - Number of observations (rows) and features (columns)
  - Brief description of the dataset and problem context
  - Definition of the target variable (e.g., default vs fully paid)
- Data Quality Assessment
  - Identification of missing values and their frequencies
  - Noted patterns in missingness (random vs systematic)
  - Detection of extreme or unusual values (outliers)
  - Data type inconsistencies or formatting issues
- Feature Summaries
  - Descriptive statistics for numeric features (mean, median, min, max, quartiles)
  - Identification of skewed or heavy-tailed distributions
  - Summary of categorical feature levels and rare categories
- Target Variable Analysis
  - Counts and proportions of each target class
  - Assessment of class imbalance
  - Potential implications of imbalance for modeling
- Relationships Between Features
  - Correlations among numeric features
  - Comparisons of feature distributions across target classes
  - Identification of features showing early predictive signals
- Key Observations & Hypotheses
  - 1–3 concise insights derived from EDA
  - Clear, testable hypotheses (e.g., "Higher DTI is associated with increased default risk")
- Visualizations
  - At least one EDA visualization (histogram, boxplot, bar chart, heatmap, etc.)
  - 2–3 sentence interpretation explaining what the plot shows and why it matters

Data Cleaning

What Data Cleaning Is and Why It Matters

- Data cleaning is the process of detecting, correcting, or removing inaccurate, incomplete, inconsistent, or irrelevant data
- It typically takes 60–80% of a real data science project
- Bad data → misleading EDA → garbage models → wrong conclusions
- Clean data does not mean "perfect" — it means fit for purpose

High-Level Goals of Data Cleaning

- Ensure data is accurate
- Ensure data is consistent
- Ensure data is complete enough for analysis
- Ensure data is interpretable
- Ensure data reflects real-world meaning

First Pass: Structural Checks

- Dataset Shape
    - Number of rows and columns
    - Unexpected changes in size (duplicates, truncation)
- Column Names
    - Clear, descriptive, consistent naming
    - Avoid spaces, mixed casing, special characters
    - Rename ambiguous variables early
- Data Types
    - Ensure numeric columns are numeric
    - Dates should be datetime objects
    - Categorical variables should not be numeric IDs by accident
- Example issues:
    - "10%" stored as string instead of numeric
    - Dates stored as text
    - Binary variables stored as "Yes" / "No" inconsistently

Missing Data

- Identify Missing Values
    - Count missing values per column
    - Look for patterns (entire rows missing, specific groups missing)
    - Watch for disguised missing values:
        - "NA", "N/A", "unknown", -999, 0 (when 0 is impossible)

- Types of Missingness
    - MCAR (Missing Completely at Random)
    - MAR (Missing at Random, conditional on other variables)
    - MNAR (Missing Not at Random — dangerous)
- How to Handle Missing Data
    - Drop rows (only if small and random)
    - Drop columns (if mostly missing or irrelevant)
    - Impute:
        - Mean / median (numeric)

        - Mode (categorical)
        - Group-based imputation
        - Model-based imputation
    - Create missingness indicators (often very powerful)

TIP: Never blindly impute, always justify the choice

Duplicate Data

- Identify Duplicates
    - Exact duplicate rows
    - Duplicate IDs
    - Near-duplicates (same entity, slight differences)
- Decisions
    - Remove exact duplicates
    - Aggregate duplicated entities if needed
    - Keep duplicates if they represent real repeated events

Outliers & Extreme Values

- Identify Outliers
    - Boxplots
    - Z-scores
    - IQR method
    - Domain-based thresholds
- Decide What They Are
    - Data errors (e.g., negative income)
    - Rare but valid observations
    - Measurement issues
- How to Handle Them
    - Remove (only if clearly erroneous)
    - Cap / winsorize
    - Transform (log, sqrt)
    - Leave them if they are meaningful

TIP: Removing outliers without justification is a red flag

Invalid, Impossible, or Inconsistent Values

- Examples:
    - Age = -5
    - Credit score = 1200
    - Loan term = 7 months when only 36 or 60 allowed

- Actions:
    - Fix if possible
    - Replace with missing
    - Drop affected rows

Categorical Data Cleaning

- Common Issues
    - Inconsistent labels ("Male", "male", "M")
    - Misspellings
    - Rare categories
    - High-cardinality variables
- Fixes
    - Standardize category names
    - Combine rare categories into "Other"
    - Verify categories align with domain knowledge

Numeric Data Cleaning

- Check Distributions
    - Skewness
    - Heavy tails
    - Zero inflation
- Transformations
    - Log transforms
    - Scaling (standardization / normalization)
    - Binning (carefully)

Target Variable Cleaning

- Ensure target is correctly labeled
- Check for:
    - Class imbalance
    - Data leakage
    - Inconsistent definitions
- Examples of leakage:
    - Features that directly encode the target
    - Post-outcome variables included in predictors
- Data Leakage Prevention
    - Ask:
        - Would this variable be known at prediction time?
        - Is this feature derived from future information?

        - Does it encode the label indirectly?

Leakage = inflated performance + failed real-world models

Consistency Across Features

- Units must match (USD vs thousands)
- Time windows must align
- Ratios must make sense
- Example:
    - Monthly income vs annual loan amount → fix or convert

Validation Checks (Sanity Tests)

- Summary statistics make sense
- Ranges are realistic
- Relationships follow logic
- No impossible combinations
- Example:
    - Someone marked as “fully paid” with zero payments made → suspicious

Documentation

- You should document:
    - What was cleaned
    - Why decisions were made
    - What assumptions were used
    - What data was dropped or altered
- This protects:
    - You
    - Your analysis
    - Your grade / credibility

When to Stop Cleaning

- You stop when:
    - Data supports the project goal
    - Major errors are resolved
    - Assumptions are documented
    - Remaining noise reflects reality, not mistakes

TIP: Clean ≠ perfect

How Data Cleaning Fits in the Pipeline

1. Raw data ingestion
2. Initial EDA
3. Data cleaning
4. Feature engineering
5. Modeling
6. Evaluation
7. Iteration (cleaning may repeat)

Things to Clean for Our Dataset:

- **Check for duplicate rows**
- **Missing values in 'ChgOffDate'**
    - **Drop the column 'ChgOffDate'**
- **Dollar signs '$' in a couple of variables: 'ChgOffPrinGr', 'GrAppv' "SBA_Appv"**
    - **Get rid of the dollar signs and make sure the datatype is numerical**
- **NAICS is all numerical**
    - **Convert to categorical**
- **Columns that don't seem useful**
    - **Drop columns 'ApprovalDate', 'LoanNr_ChkDgt', 'Name',**
- **'MIS_Status' has missing variables**
    - **Drop the rows with missing NA 'MIS_Status'**
- **Scale numeric data (Sarah)**
    - **Changing some numerical to string or vice versa**
- **Encode categorical variables**

Feature Engineering

What Feature Engineering Is (Big Picture)

- Feature engineering is the process of creating, transforming, and selecting variables to improve model performance
- It encodes domain knowledge into the data
- Often more impactful than model choice
- Great features + simple model > complex model + weak features

Goals of Feature Engineering

- Increase predictive signal
- Reduce noise
- Improve model interpretability
- Make relationships more linear or learnable
- Prevent data leakage

- Improve generalization

When Feature Engineering Happens

- After initial data cleaning
- During EDA (hypotheses → features)
- Iteratively after modeling results
- Never just once, it rotates n a cycle

Core Types of Feature Engineering

Feature Creation (New Features)

- You combine existing variables to create new information.
- Examples:
    - Ratios: debt / income
    - Differences: current_balance - previous_balance
    - Aggregates: mean, sum, max over time
    - Flags: is_high_risk = (DTI > 40)

TIP: Ratios and differences are huge in real projects.

Feature Transformation

- Change the scale or distribution.
- Common transforms:
    - Log (log(x + 1))
    - Square root
    - Box-Cox / Yeo-Johnson
    - Standardization (z-score)
    - Min-max scaling
- Why:
    - Reduce skew
    - Handle heavy tails
    - Help linear and distance-based models

Feature Encoding (Categorical Variables)

1. One-Hot Encoding
    - Best for low-cardinality categorical features
    - Avoid dummy variable trap if needed
2. Label Encoding
    - Only when order exists
    - Dangerous otherwise
3. Target / Mean Encoding

- Replace category with average target value
- Must use cross-validation to avoid leakage

4. Frequency Encoding
   - Replace category with its frequency

Feature Binning / Bucketing

- Turn continuous variables into categories.
- Examples:
    - Age groups
    - Income brackets
    - Risk tiers
- Pros:
    - Robust to outliers
    - Improves interpretability
- Cons:
    - Loses information if done poorly

Interaction Features

- Capture combined effects.
- Examples:
    - income × employment_length
    - loan_amount / income
    - DTI × credit_score

- Especially useful for:
    - Linear models
    - Logistic regression

TIP: Tree models can learn interactions automatically — but engineered ones still help.

Feature Engineering by Data Type

- Numeric Data
    - Scaling
    - Log transforms
    - Ratios
    - Polynomial terms (carefully)
    - Outlier flags
- Categorical Data
    - Encoding

    - Rare category grouping
      Domain-driven grouping
- Date/Time Data
    - Year, month, day
    - Day of week
    - Time since event
    - Seasonality indicators
- Text Data
    - Length
    - Word counts
    - TF-IDF
    - Sentiment scores

Feature Selection (What to Keep)

- Why Feature Selection Matters
    - Reduces overfitting
    - Improves interpretability
    - Speeds training
- Methods
    - Filter Methods
    - Correlation thresholds
    - Mutual information
    - Variance thresholding
- Wrapper Methods
    - Recursive Feature Elimination (RFE)
- Embedded Methods
    - Lasso
    - Tree-based feature importance

TIP: Correlation ≠ useless. Correlated features can still help.

Feature Engineering for Specific Models

- Linear / Logistic Regression
    - Scaling is critical
    - Interactions help
    - Log transforms improve linearity
- Tree-Based Models
    - Less sensitive to scaling
    - Can handle non-linearities
    - Still benefit from:

        - Clean categorical encoding
        - Strong ratios
        - Aggregated features
- Distance-Based Models (KNN, SVM)
    - Scaling is mandatory
    - Feature selection is crucial

Feature Engineering & Class Imbalance

- Create features that separate minority class
- Use class-specific aggregations
- Avoid features that simply mirror imbalance

Feature Leakage (Biggest Mistake)

- Ask for every feature:
    - Would this be known at prediction time?
    - Is it derived from future information?
    - Is it a proxy for the target?
- Examples:
    - "Total payments made" predicting loan default
    - Status codes that encode the label

TIP: Leakage = fake performance

Feature Validation & Evaluation

- Check Feature Quality
    - Distribution plots
    - Relationship with target
    - Stability across train/test
    - Importance scores
- Drop Features That:
    - Add noise
    - Cause leakage
    - Are redundant
    - Don't generalize

Feature Engineering Workflow

1. Understand the domain
2. EDA → hypotheses
3. Create candidate features
4. Validate features

5. Select features
6. Train model
7. Iterate

Common Feature Engineering Mistakes

- Blindly adding features
- Over-engineering
- Ignoring domain knowledge
- Creating leakage
- Not validating features
- Forgetting interpretability

Action Items for Next Week:

- Re-run final EDA on the cleaned dataset and confirm summary statistics, distributions, and correlations
- Resolve any remaining missing values, outliers, and data type inconsistencies
- Standardize and validate categorical variables; remove duplicates or irrelevant features
- Finalize and verify all engineered features; remove redundant or collinear features
- Re-check target variable correctness and absence of data leakage
- Perform final sanity and consistency checks across all features
- Lock the final cleaned dataset and feature set for modeling
- Document all cleaning and feature engineering decisions

Week 6 Meeting

Week 6: Final EDA and Modeling

Data Cleaning and Feature Engineering Check-In:

- Are all duplicate rows removed?
- Are the columns ‘ChgOffDate’, ‘ApprovalDate’, ‘LoanNr_ChkDgt’, ‘Name’ dropped?
- Are the dollar signs in ‘GrAppv’, ‘SBA_Appv’ removed and those columns converted to numerical datatypes?
- Is NAICS converted to categorical?
- Are the rows with missing or NULL ‘MIS_Status’ dropped?
- Is the numeric data scaled?
- Are the categorical variables encoded?

Building the Model

What is a Model?

A model is like a prediction machine. You feed it historical information (the past), and it learns the patterns so it can guess what will happen next (the future).

- Inputs (X): The clues we give the machine (e.g., a person’s credit score or income).
- Target (y): The answer we want the machine to guess (e.g., "Will they pay back this loan?").

How the Machine Learns

Think of training a model like teaching a student using a practice test.

1. The Guessing Game: At first, the machine knows nothing. It makes a random guess.
2. The "Correction" (Loss Function): We compare the machine's guess to the real answer. If it’s wrong, we tell it how far off it was.
3. Turning the Knobs (Optimization): The machine has "knobs" (called weights) for every clue. If it sees that "High Income" usually means "Pays Back Loan," it turns that knob up. This will weigh the features with higher importance higher, and the features with lower importance lower.
4. Practice Makes Perfect: The machine goes through the data thousands of times until it stops making big mistakes. This is called Convergence.

Hyperparameters

If training is the student studying, hyperparameters are the study conditions you set for them.

1. The Learning Rate (The Step Size)
   This is arguably the most important hyperparameter. It tells the machine how much to turn the "Weight Knobs" every time it makes a mistake.
   - If it's too high: The machine is too aggressive. It turns the knobs so hard that it overshoots the correct answer. It’s like trying to park a car by slamming the gas and then slamming the brakes.

    - If it's too low: The machine is too timid. It turns the knobs by a fraction of a millimeter. Training will take forever, and the machine might get stuck before it ever finds the right answer.
2. Number of Estimators (The Size of the Team)
   In "Ensemble" models (like Random Forest or XGBoost), this is the number of individual trees the machine is allowed to build.
    - The Logic: If you have 100 trees "voting" on a loan, you get a more stable answer than if you only have 3 trees.
    - The Trade-off: More trees usually mean better accuracy, but they also make the model slower and require more computer memory.
3. Max Depth (The Complexity Limit)
   This tells the machine how many "questions" it is allowed to ask for each loan application.
    - Shallow (Small Depth): The machine can only ask 2 questions (e.g., "Is income high?" and "Is credit score good?"). This might be Underfitting because it's too simple.
    - Deep (Large Depth): The machine can ask 50 questions. It might start asking useless questions like, "Was the loan signed on a Tuesday while it was raining?" This leads to Overfitting (memorizing noise).
4. Regularization (The "Penalty" for Being Too Complex)
   This is like a "Simplicity Tax." It tells the machine, "I will punish you if you try to make the weight knobs too large or too complex."
    - Why it matters: It forces the machine to focus only on the most important clues and ignore the small, distracting details. It is a great tool to fight overfitting.

How do we find the "Right" Hyperparameters?
Since the machine can't learn these, we use a process called Hyperparameter Tuning.

- Grid Search: We give the machine a list of options (e.g., Learning Rate = 0.1, 0.01, or 0.001) and tell it to try every single combination until it finds the one that gets the best grade on the practice test. It takes a long time and is very extensive but is guaranteed to find the best combination of hyperparameters.
- Random Search: The machine picks random combinations of settings to try to find a "good enough" configuration quickly. Can find the hyperparameters faster but might not always be the most optimal combination.

The Train-Test Split (The Golden Rule)
You can’t test a student using the exact same questions they just practiced. They would just memorize the answers! To be fair, we split our data into two piles:

- The Training Pile (80%): The "Practice Test." The machine uses this to learn the patterns.

- The Testing Pile (20%): The "Final Exam." The machine has never seen these specific examples. This is how we prove it actually learned something and didn't just memorize.

Algorithms to Train Models on

Train these algorithms

- Logistic Regression - Jessica Kim
- Decision Tree Classification - Kevin Tran
- Random Forest Classification - Lucy Guo
- KNN Classification - Chloe Lee
- XGBoost Classification - Sarah Dias

Action Items for Next Week:

- Create a final EDA plot on the cleaned data with an explanation on what it is showing and what trend there is, make sure there is specific impact with the insight
- Learn about and train a model on your assigned algorithm, train the hyperparameters
- Prepare to teach the rest of us how your assigned algorithm works

Week 7 Meeting

Week 7: Model Interpretation & Insights

Final EDA and Modeling Check-In:

- Check in on final EDA plots
- Check in on initial model building

Algorithm Teaching

- Logistic Regression (Jessica Kim)
- Decision Tree Classification (Kevin Tran)
- Random Forest Classification (Lucy Guo)
- KNN Classification (Chloe Lee)
- XGBoost Classification (Sarah Dias)

Model Evaluation

Accuracy is a Trap

If 99% of people are healthy, a doctor who says "Everyone is healthy" is 99% accurate but misses the 1% who are sick.

- Precision:
    - It is the ratio of True Positives (correct positive predictions) to the total number of positive predictions made by the model (which includes both correct predictions and "false alarms").
    - The Math: Precision = True Positives / (True Positives + False Positives)
    - The Goal: To avoid False Alarms (False Positives).
    - Primary metric used when the cost of a false positive is high. A high precision score would indicate a low false positive rate, so when the model predicts a specific outcome, that outcome is highly reliable.
- Recall:
    - It is the ratio of True Positives (correct positive predictions) to the total number of actual positives in the dataset (which includes both the ones the model caught and the "missed dangers").
    - The Math: Recall = True Positives / (True Positives + False Negatives)
    - The Goal: To avoid Missed Dangers (False Negatives).
    - Primary Metric: Used when the cost of a false negative is high. A high recall score indicates that the model is successfully capturing most of the positive cases, making it a reliable "safety net" for identifying risk.
- F1 Score
    - It is the harmonic mean of Precision and Recall, providing a single score that accounts for both metrics simultaneously.
    - The Math: F1 Score = 2 * (Precision * Recall) / (Precision + Recall)
    - The Goal: To find a Balance between Precision and Recall.

    - Primary Metric: Used when you have an uneven class distribution (imbalanced data) and you need a balance between avoiding false alarms and catching all positive cases. It is the best "all-around" grade because it punishes extreme values, if either Precision or Recall is very low, the F1 score will also be low.

Underfitting vs. Overfitting

We want our model to be "just right."

- Underfitting (Too Simple): The machine is too lazy. It doesn't find any patterns. Errors are caused by overly simplistic assumptions. The model misses the relevant relations between features and target outputs. It's like a student who didn't study at all.
- Overfitting (Too Complex): The machine tries too hard. It memorizes random details that don't matter. Errors are caused by excessive sensitivity to small fluctuations in the training set. It does great on the practice test but fails the final exam.

K-Fold Cross-Validation

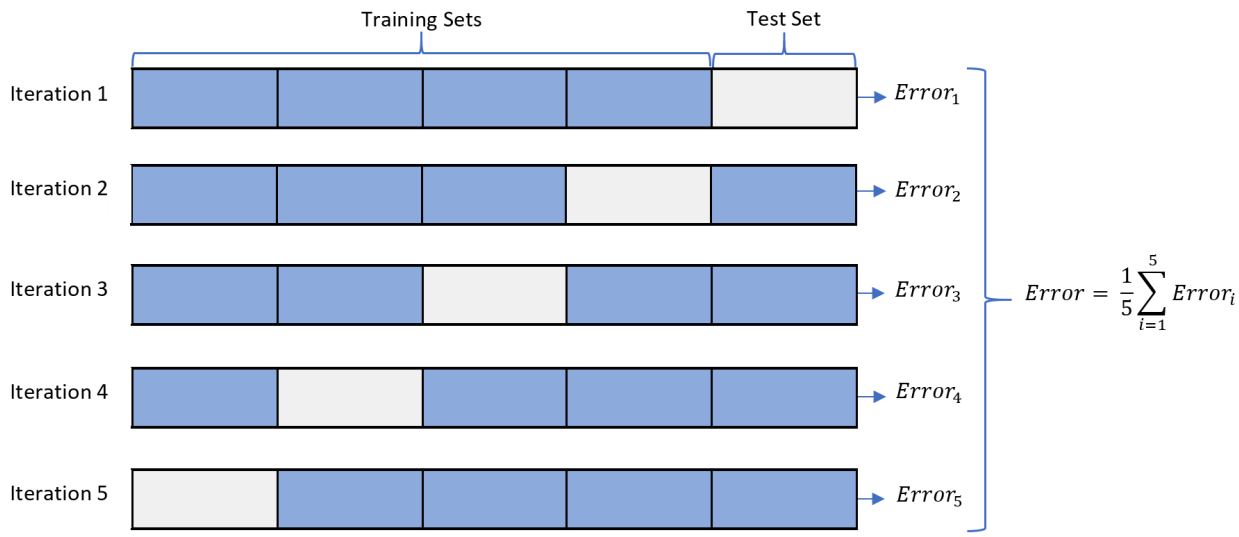
To ensure a model hasn't just "memorized" a specific set of data or gotten "lucky" with an easy test, we use a rotation strategy called K-Fold Cross-Validation.

The Process

1. Split: The entire dataset is divided into K equal groups (called Folds). Usually, K is set to 5 or 10.
2. Rotate: The model is trained K different times. In each round, a different fold is held out as the "Final Exam" (Test Set), while the remaining folds are used as the "Study Guide" (Training Set).
3. Average: After all K rounds are complete, every piece of data has been used for both training and testing. We average the scores from each round to calculate the "True Grade."

Why It Matters

- Eliminates Luck: It prevents a "lucky split" where the test set happened to be full of easy-to-predict examples.
- Maximizes Data: It allows the model to be tested against every single row of data you have, providing a much more honest view of how it will perform in the real world.
- Consistency: If the scores across all K folds are very different, it's a red flag that the model is unstable or the data is inconsistent.



Data Leakage

The Two Main Types of Leakage

1. Target Leakage
   This occurs when your predictors include data that is a "proxy" for the answer or only exists after the event you are trying to predict has already happened.
   - The Example: If you are building a model to predict if a customer will default on a loan, and you include a column for "Late Fees Collected," you have leaked the answer. You only collect late fees after a person begins to default.
   - The Result: The model "cheats" by looking at the fees to guess the default, rather than learning the actual risk factors (like income or credit history).
2. Train-Test Contamination
   This happens when information from the Test Set (the "Final Exam") is accidentally shared with the Training Set (the "Study Guide") before the model is finished.
   - The Example: You calculate the "Average Income" of your entire dataset and use that number to fill in missing values before you split the data into training and testing.
   - The Result: Your training data now contains a "summary" of your test data. The model has essentially seen a "sneak peek" of the final exam questions.

Why It Ruins Evaluation

Data leakage creates a False Sense of Confidence.

- During Evaluation: You might see a 99% accuracy or a perfect F1-Score. You think you've built the best model in history.
- In the Real World: When you try to predict a new loan application, you don't have the "Late Fees" or the "Future Average Income" yet. The model, which relied on those "leaked" clues, will now struggle to make any sense of the data.

How to Prevent It

1. Check the Timeline: For every feature, ask: "Would I know this value at the exact moment I need to make the prediction?" If the answer is no, drop it.
2. Split Early: Always split your data into Training and Testing before you do any cleaning, scaling, or math.
3. Audit High Correlations: If a feature is almost perfectly correlated with the target, investigate it. It's often a leaked version of the answer.

Confusion Matrix

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Detailed Category Breakdown

1. True Positive (TP): The "Bullseye"
   - In Credit Risk: You predicted the loan would default, and it did.
   - Impact: You successfully avoided a loss. High TP counts contribute to high Recall.
2. True Negative (TN): The "Silent Success"
   - In Credit Risk: You predicted the borrower would pay, and they did.
   - Impact: The bank makes money on interest. This is the "ideal" business transaction.
3. False Positive (FP): The "Type I Error" (False Alarm)
   - In Credit Risk: You flagged a good borrower as a "Default" risk and rejected them.
   - The Business Cost: You "insulted" a good customer. You lost their business to another bank and lost the interest you would have earned.
   - Connection: This is the enemy of Precision.
4. False Negative (FN): The "Type II Error" (Missed Danger)

- In Credit Risk: You predicted a borrower was safe, but they defaulted.
- The Business Cost: This is the "catastrophic" error. The bank loses the entire principal of the loan ($10k, $50k, etc.).
- Connection: This is the enemy of Recall.

Model Insights

Coefficient Analysis (Logistic Regression)

Logistic Regression is the most "transparent" model. It gives you an exact mathematical weight for every feature.

- The Insight: Each coefficient tells you how much the log-odds of the outcome (e.g., defaulting) change with a one-unit increase in the feature.
- The Goal: Identifying "Risk Factors" vs. "Protective Factors." A positive coefficient increases the probability of the event, while a negative one decreases it.
- Take a look at the coefficients of the Logistic Regression Model

Feature Importance (Decision Trees, Random Forest, & XGBoost)

These "Tree-Based" models provide a ranking of features based on their Gini Importance or Gain.

- How it works: The model tracks which features were used to make the most impactful "splits" in the data. Features at the top of the trees that consistently separate "Good" from "Bad" borrowers get the highest scores.
- Why it Matters: While a Random Forest is a "Black Box" (you can't easily read 100+ trees), the Feature Importance Plot summarizes their collective wisdom into a simple bar chart.
- Use model.feature_importances_

Feature Selection & Sensitivity (KNN)

KNN is a "Lazy Learner" and does not have a built-in "Feature Importance" score because it doesn't learn a global rule; it just looks at neighbors.

- The Insight: To understand KNN, you must look at Feature Scaling. If you don't scale your data, the feature with the largest numbers (like "Annual Income") will completely dominate the "distance" calculation, making other features (like "Years of Credit History") invisible.
- The Strategy: We gain insight here by dropping features one by one and seeing how much the accuracy falls. If accuracy drops significantly when a feature is removed, that feature is a "key neighbor" trait.
- Use sklearn.inspection

Action Items for Next Week:

- Optimize the model after evaluating the model, and then develop 2-3 key insights about what drives credit risk and defaulting

TIP: Keep in mind our dataset is imbalanced so make sure to come up with a solution for that

Week 8 Meeting

Week 8: Presentation and Dashboard Building

Model Interpretation & Insights Check-In:

- Check in on final model and 2-3 model insights from each person

Project Showcase

The project showcase is set for **March 3rd, starting at 7pm in Dodd Hall 161**, mark in your calendars **DON'T MISS IT!**

Business Casual Attire!

Presentation

Let's also build our presentation

Possible Themes:
https://slidesgo.com/theme/banking-infographics#search-credit+risk&position-4&results-14&rs=search
https://slidesgo.com/theme/community-bank-business-plan#search-credit+risk&position-6&results-14&rs=search
https://slidesgo.com/theme/investors-social-media#search-credit+risk&position-5&results-14&rs=search

Sections:

- Project Problem & Introduction (Jessica)
- EDA & Preprocessing (Kevin) (ANYONE CAN ADD THEIR OWN EDA CHARTS AS WELL)
- Model (EVERYONE)
- Model Comparisons (Chloe)
- Model Insights (Sarah)
- Summary (Lucy)
    - Summary of our process (very short)
    - Summary of our insights from EDA
    - Short reminder of what our best model was
    - Summary of our insights from Model insights

NEEDS TO BE DONE BY SATURDAY, FEBRUARY 28th

Action Items for Next Week:

- *Finish the presentation and be ready for presenting Tuesday!*

- *See you all in* ***Business Casual Attire at Dodd Hall 161 on March 3rd at 7pm****!*
- **Monday night 9pm ZOOM Run through**

Week 9

Week 9: Project Showcase

Presentation
NSDC Credit Risk Presentation

Presentation Order
Section 1 (Introduction): Philbert
Section 2 (EDA & Preprocessing): Kevin
Section 3 (ML Models): Jessica
Section 4 (Model Comparisons): Chloe
Section 5 (Final Model Insights): Sarah
Section 6 (Summary): Lucy

Code Notebook
CreditRisk_NSDCProject.ipynb

Thank You!!
Thank you for all your hard work this quarter!!